Autor:
Ana Isabel Vegas

ibertech

end2end dedication

CURSO

Introducción al Testing

www.ibertech.es

ibertech

end2end dedication

Tema 1 Fundamentos de testing

Tema 2 Configuración

Tema 3 Tipos de pruebas y conceptos principales

Tema 4 Test avanzados

Tema 5 TDD – Test Driven Development

Tema 6 BDD – Behavior Driven Development

ibertech

end2end dedication

Tema 1

Fundamentos de Testing

Qué son las pruebas de código?

- Las pruebas de código (o tests de software) son procedimientos que se usan para verificar que un programa o una parte del código funcione correctamente, según lo esperado. En otras palabras, sirven para detectar errores (bugs), asegurar la calidad del software y comprobar que los cambios no rompan funcionalidades existentes.

Por qué hacer pruebas?

- Las pruebas ayudan a:
  - Detectar errores o bugs en el programa.
  - Comprobar que las funciones hacen lo que deberían.
  - Evitar que cambios nuevos rompan partes del sistema que ya funcionaban.
  - Mejorar la calidad y confiabilidad del software.

Como escribir código que se pueda probar

- Escribir código que se pueda probar fácilmente (código *testable*) significa diseñarlo de forma que puedas verificar su comportamiento con pruebas automáticas sin dificultad.
- Esto se logra siguiendo algunos principios de diseño:

1. Funciones pequeñas y con una sola responsabilidad
    - Cada función debería hacer solo una cosa.
2. Evitar dependencias ocultas
    - El código es más fácil de probar cuando recibe lo que necesita como parámetros en lugar de usar variables globales o recursos externos directamente.
3. Separar lógica de efectos externos
    - La lógica del programa debería estar separada de cosas como: base de datos, APIs, archivos, red, interfaz gráfica

Como escribir código que se pueda probar

4. Usar entradas y salidas claras
   - Las funciones deberían recibir datos claros y devolver resultados claros
5. Evitar demasiada lógica en un solo lugar
   - Cuando una función tiene muchas condiciones es más difícil probarla.
   - Divide en funciones más pequeñas.
6. Usar interfaces o abstracciones
   - Esto permite reemplazar componentes reales por simulados en las pruebas.

Frameworks de pruebas

- Los frameworks de pruebas (testing frameworks) son herramientas que ayudan a automatizar la comprobación de que un programa funciona correctamente. Permiten escribir tests que ejecutan partes del código y verifican si el resultado es el esperado. Son fundamentales en desarrollo de software para evitar errores y mantener la calidad del código.

- Un framework de pruebas permite:
  - Automatizar pruebas de código
  - Detectar errores temprano en el desarrollo
  - Verificar cambios cuando se modifica el código
  - Facilitar el desarrollo guiado por pruebas (TDD)
  - Generar reportes de resultados de pruebas

Tipos de Frameworks de pruebas

1. Pruebas unitarias: Prueban pequeñas partes del código (funciones o métodos).

- JUnit (Java)
- pytest (Python)
- NUnit (C#)
- Jest (JavaScript)

2. Pruebas de integración: Comprueban que varios componentes del sistema funcionan bien juntos.

- TestNG
- Mockito

3. Pruebas funcionales / end-to-end: Simulan acciones reales de un usuario en la aplicación.

- Selenium
- Cypress
- Playwright

Principios de diseño

- Los principios de diseño en pruebas de software son reglas que ayudan a crear pruebas efectivas, mantenibles y que detecten errores de forma eficiente. Estos principios guían cómo planificar y ejecutar las pruebas durante el desarrollo del software.

1. **Las pruebas muestran la presencia de defectos**

- Las pruebas no pueden demostrar que el software está libre de errores, solo pueden demostrar que existen defectos.
- Este principio es parte de los fundamentos del testing definidos por organizaciones como International Software Testing Qualifications Board.

2. **Pruebas exhaustivas son imposibles**

- No es posible probar todas las combinaciones de entradas y escenarios de un sistema.
- Por eso se utilizan técnicas de selección de casos de prueba, priorizando los más importantes o de mayor riesgo.

Principios de diseño

3. Pruebas tempranas

- Las pruebas deben empezar lo antes posible en el ciclo de desarrollo.
- Detectar errores en etapas tempranas reduce costos y tiempo de corrección.

4. Agrupación de defectos

- La mayoría de los errores suelen encontrarse en un pequeño número de módulos del sistema.
- Esto se relaciona con el principio de Pareto, formulado por Vilfredo Pareto.

5. Paradoja del pesticida

- Si se ejecutan siempre las mismas pruebas, con el tiempo dejan de encontrar nuevos errores.
- Por eso los casos de prueba deben revisarse y actualizarse regularmente.

6. Las pruebas dependen del contexto

- No todas las aplicaciones requieren el mismo tipo de pruebas.
- Por ejemplo:
  - Un sistema médico necesita pruebas más estrictas.
  - Un videojuego puede priorizar pruebas de experiencia de usuario.

7. Falacia de ausencia de errores

- Que un sistema no tenga fallos detectados no significa que sea útil o que cumpla con las necesidades del usuario.
- Un software puede funcionar perfectamente pero resolver el problema equivocado.

ibertech

end2end dedication

Tema 2

Configuración

Preparación del entono de pruebas

- La preparación del entorno de pruebas es el proceso de configurar todos los recursos necesarios para ejecutar pruebas de software en condiciones controladas. El objetivo es que el sistema probado funcione en un ambiente lo más parecido posible al entorno real de producción.
- Los pasos a seguir son:

    1. Definir el entorno de pruebas
    2. Configurar hardware y software
    3. Instalar la aplicación a probar
    4. Preparar datos de prueba
    5. Configurar herramientas de prueba
    6. Verificar el entorno
    7. Documentar el entorno

1. Definir el entorno de pruebas

- Se determina qué tipo de entorno se necesita, por ejemplo:
    - Entorno de desarrollo
    - Entorno de pruebas o testing
    - Entorno de preproducción
    - Entorno de producción

- Cada uno tiene configuraciones diferentes según el tipo de pruebas que se van a realizar.

2. Configurar hardware y software

- Se preparan los recursos necesarios:
  - **Hardware**
    - Servidores
    - Computadoras de prueba
    - Dispositivos móviles (si es necesario)
  - **Software**
    - Sistema operativo
    - Bases de datos
    - Servidores web
    - Navegadores

3. Instalar la aplicación a probar

- Se despliega la aplicación en el entorno de pruebas y se configuran sus dependencias:
  - Servicios externos
  - APIs
  - Bases de datos

- Esto permite que el sistema funcione correctamente durante las pruebas.

4. Preparación datos de pruebas

- Se crean datos específicos para las pruebas, por ejemplo:
  - Usuarios de prueba
  - Registros en bases de datos
  - Datos simulados o anonimizados

- Estos datos permiten ejecutar diferentes escenarios de prueba.

5. Configurar herramientas de prueba

- Se instalan herramientas que ayudan a ejecutar o automatizar pruebas, como:
  - Selenium
  - Cypress
  - JUnit

6. Verificar el entono

- Antes de comenzar las pruebas completas, se ejecutan pruebas básicas para comprobar que el entorno funciona correctamente.
- Esto asegura que:
  - La aplicación se ejecuta
  - Los servicios responden
  - La base de datos funciona

7. Documentar el entono

- Finalmente se documenta:
  - Configuración del sistema
  - Versiones de software
  - Herramientas instaladas
  - Procedimiento de instalación

- Esto facilita reproducir el entorno en el futuro.

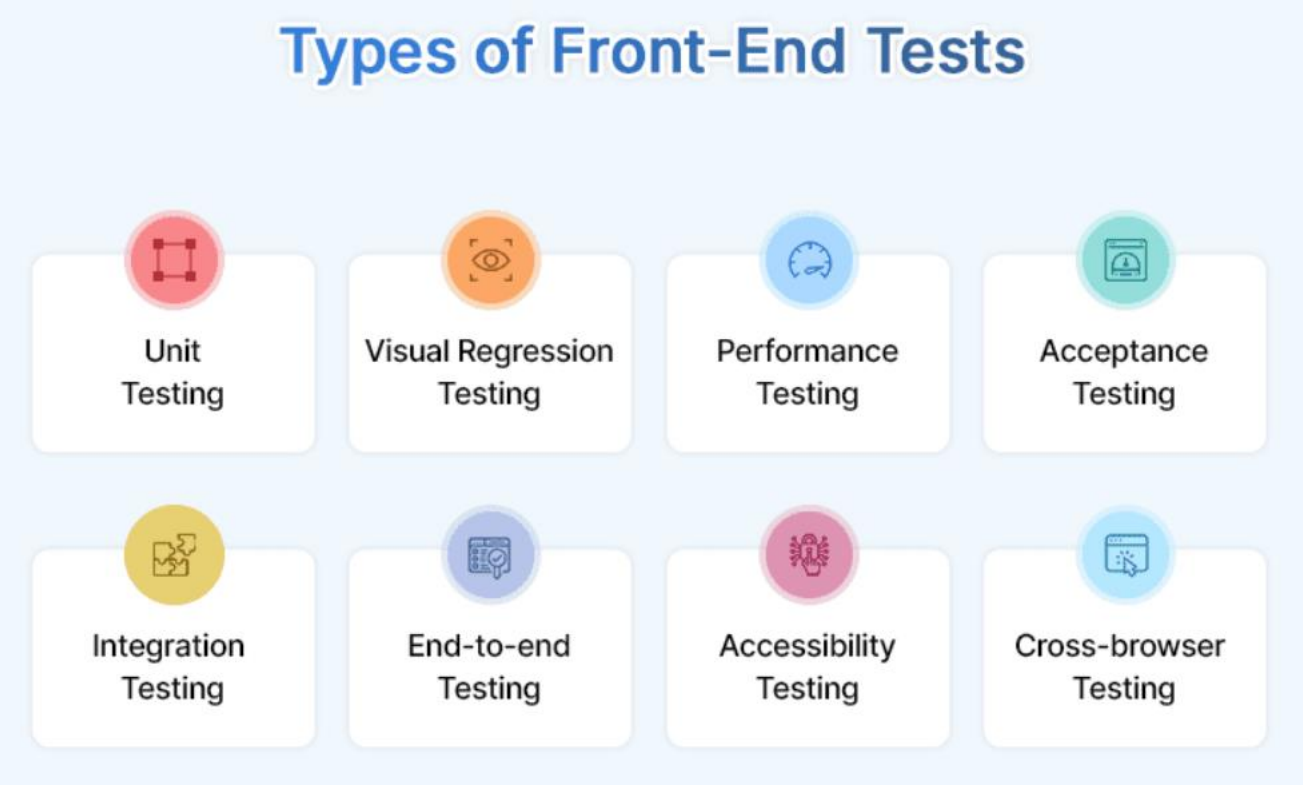
ibertech

end2end dedication

Tema 3

Tipos de pruebas y conceptos principales

Tipos de pruebas



Pruebas esenciales

- **Pruebas unitarias**: Verifican pequeñas unidades de código, como funciones, métodos o componentes individuales, de forma aislada para asegurar que funcionan correctamente por sí solas.
- **Pruebas de integración**: Evalúan cómo interactúan varios componentes o módulos al trabajar juntos, asegurando que sus conexiones y colaboraciones sean correctas.
- **Pruebas funcionales/end-to-end (E2E)**: Simulan el comportamiento real del usuario y prueban flujos completos dentro de la aplicación, asegurando que todo el sistema funcione según lo esperado desde la perspectiva del usuario final.
- **Pruebas de regresión**: Comprueban que los cambios recientes en el código no hayan roto funcionalidades que antes funcionaban bien.
- **Pruebas visuales**: Detectan cambios inesperados en la interfaz gráfica comparando capturas de pantalla antes y después de una modificación.

Otras pruebas importantes

- **Pruebas entre navegadores (cross-browser):** Garantizan que la aplicación funcione correctamente en distintos navegadores, sistemas operativos y dispositivos.
- **Pruebas de rendimiento:** Evalúan la velocidad, capacidad de respuesta y estabilidad de la aplicación bajo diferentes condiciones de uso.
- **Pruebas de accesibilidad:** Se utilizan para evaluar y optimizar la interfaz de usuario de una aplicación web para garantizar que sea utilizable por personas con diferentes discapacidades, ya sean visuales, auditivas, motoras o cognitivas.

Prueba Unitaria

- Una prueba unitaria es una forma de probar el correcto funcionamiento de un módulo de código. Esto sirve para asegurar que cada uno de los módulos funcione correctamente por separado.

- La idea es escribir casos de prueba para cada función no trivial o método en el módulo de forma que cada caso sea independiente del resto. Esto último es la esencia de una prueba unitaria: se prueba al componente de forma aislada a todos los demás.

Características de las pruebas unitarias

- Para que una prueba unitaria sea buena se deben cumplir los siguientes requisitos:
  - **Automatizable**: no debería requerirse una intervención manual. Esto es especialmente útil para Integracion Continua.
  - **Completas**: deben cubrir la mayor cantidad de código.
  - **Repetibles o Reutilizables**: no se deben crear pruebas que sólo puedan ser ejecutadas una sola vez. También es útil para Integracion Continua.
  - **Independientes**: la ejecución de una prueba no debe afectar a la ejecución de otra.
  - **Profesionales**: las pruebas deben ser consideradas igual que el código, con la misma profesionalidad, documentación, etc.

Ventajas en el uso de Pruebas Unitarias

- **Detección temprana de errores**: Identifican problemas en etapas iniciales del desarrollo.
- **Facilitan el mantenimiento**: Aseguran que los cambios en el código no rompan funcionalidades existentes.
- **Documentación viviente**: Sirven como una guía clara sobre cómo debería comportarse cada unidad.

Pruebas de Unidad e Integración en JavaScript

Pruebas unitarias y de integración

- Las pruebas unitarias en JavaScript consisten en verificar que una unidad individual del código (como una función o método) funcione correctamente aislada del resto del sistema. Por ejemplo, se prueba que una función suma correctamente como unidad independiente.

- Las pruebas de integración van un paso más allá, asegurando que varias unidades de código funcionen correctamente juntas, verificando la interacción entre componentes.

Frameworks utilizados

- Para realizar tanto pruebas unitarias como de integración en JavaScript, se utilizan comúnmente frameworks como Jest, Mocha, Jasmine, Karma, entre otros. Jest es uno de los más populares y fáciles de usar, proporcionando una experiencia de "configuración cero" y soporte para pruebas unitarias y de integración. Mocha es otra opción flexible especialmente cuando se necesita escribir pruebas asíncronas. Estos frameworks ofrecen herramientas para escribir pruebas, hacer aserciones, simular dependencias y automatizar la ejecución de pruebas.

Herramientas principales

- **Jest**: Popular, usado por grandes empresas, soporta pruebas unitarias y de integración, fácil configuración.
- **Mocha**: Permite pruebas asíncronas, muy flexible.
- **Jasmine**: Sin necesidad de DOM, sintaxis sencilla.
- **Karma**: Entorno para ejecutar pruebas con varios frameworks.
- **QUnit**: Versátil para cliente y servidor.
- **Unit.js**: Biblioteca de aserciones compatible con otros frameworks.

Jest

- Jest es un framework de pruebas JavaScript desarrollado por Facebook, muy popular por su facilidad para escribir y correr pruebas, sin necesidad de configuraciones complicadas. Sirve para hacer pruebas unitarias, de integración y más.

sum.js

```
function suma(a, b) {
  return a + b;
}

module.exports = suma;
```

test.sum.js

```
const suma = require('./sum');

test('suma 1 + 2 es igual a 3', () => {
  expect(suma(1, 2)).toBe(3);
});
```

Sintaxis de Jest

- La sintaxis de Jest es intuitiva y está diseñada para facilitar la escritura de pruebas en JavaScript. Jest usa funciones globales principales para organizar, escribir y validar pruebas:
  - **describe(nombre, fn)**: Agrupa pruebas relacionadas dentro de un bloque con una descripción clara, útil para organizar el código de test.
  - **test(nombre, fn) o su alias it(nombre, fn):** Define un caso de prueba específico con su descripción y la función donde se ejecutan las aserciones.
  - **expect(valor):** Función que crea una expectativa sobre un valor, a la que se le aplican "matchers" para hacer aserciones.

Matchers comunes en Jest

- La sintaxis de Jest es intuitiva y está diseñada para facilitar la escritura de pruebas en JavaScript. Jest usa funciones globales principales para organizar, escribir y validar pruebas:
  - **toBe(valor):** Verifica igualdad estricta (===).
  - **toEqual(valor):** Verifica igualdad profunda de objetos o arrays.
  - **toBeTruthy():** Confirma que el valor es "truthy".
  - **toBeFalsy():** Confirma que el valor es "falsy".
  - **toContain(elemento):** Verifica que un array o string contiene un elemento.
  - **toThrow():** Usa para verificar que una función lance un error.

Hocks para configurar pruebas en Jest

- Estos hooks permiten preparar o limpiar estados antes y después de las pruebas, por ejemplo, inicializar datos o mocks:
  - **beforeEach(fn):** Código que se ejecuta antes de cada prueba.
  - **afterEach(fn):** Código que se ejecuta después de cada prueba.
  - **beforeAll(fn):** Se ejecuta una vez antes de todas las pruebas.
  - **afterAll(fn):** Se ejecuta una vez después de todas las pruebas.

Pruebas asíncronas con Jest

- Jest maneja pruebas asíncronas con funciones async/await, devolviendo Promesas, o usando callbacks con done().

```
test('prueba asíncrona con async/await', async () => {
  const data = await fetchData();
  expect(data).toBe('valor esperado');
});
```

Mocha

- Mocha es un framework de testing para JavaScript que se puede usar tanto en Node.js como en navegadores.
- Es muy flexible, permite escribir pruebas unitarias y de integración, soporta pruebas asíncronas y se puede combinar con librerías de aserción como Chai.
- Es una herramienta popular para asegurar la calidad del código en proyectos JavaScript.

```javascript
function suma(a, b) {
  return a + b;
}

module.exports = suma;  // Exportamos con CommonJS
```

```javascript
const expect = require('chai').expect;
const suma = require('../sum');  // Importamos con require

describe('Pruebas de suma', function() {
  it('debería devolver 3 al sumar 1 + 2', function() {
    expect(suma(1, 2)).to.equal(3);
  });
});
```

Jasmine

- Jasmine es un framework de pruebas unitarias para JavaScript ampliamente usado, especialmente en proyectos Angular, aunque es independiente del framework. Su objetivo es facilitar la escritura de tests con una sintaxis clara y legible que también sirve como documentación del comportamiento del código.

- Características principales:
  - No depende de otros frameworks ni del DOM, funciona en varios entornos.
  - Sigue el paradigma Behavior-Driven Development (BDD), permitiendo definir suites de pruebas con describe y casos con it.
  - Ofrece funciones para configuraciones previas (beforeEach, beforeAll) y de limpieza posterior (afterEach, afterAll).
  - Usa expect() junto con “matchers” para validar resultados, como toBe(), toEqual(), toBeTruthy(), entre otros.
  - Permite espiar funciones con spyOn() para verificar interacciones.

Ejemplo de Jasmine

```
describe('Calculadora', () => {
  let total;

  beforeEach(() => {
    total = 0;
  });

  it('suma correctamente dos números', () => {
    total = 1 + 2;
    expect(total).toBe(3);
  });

  it('resta correctamente', () => {
    total = 5 - 3;
    expect(total).toBe(2);
  });
});
```

Karma

- Karma es un *test runner* para JavaScript cuya función principal es ejecutar pruebas en diferentes navegadores de manera automática, facilitando la validación continua del código. No es un framework de pruebas por sí mismo, sino que se integra con otros frameworks como Jasmine, Mocha o QUnit para correr las pruebas escritas con ellos.

- Características:
  - Ejecuta pruebas en múltiples navegadores (Chrome, Firefox, Safari, IE, etc.).
  - Facilita la integración con entornos de desarrollo y sistemas de integración continua (CI/CD).
  - Detecta cambios en archivos y vuelve a lanzar las pruebas automáticamente.
  - Permite configurar reportes y métricas de pruebas.
  - Se utiliza comúnmente junto con Jasmine para testing en Angular, donde viene preconfigurado en Angular CLI.

Pruebas de Unidad e Integración en Java

JUnit

- JUnit es un conjunto de clases (framework) que permite realizar la ejecución de clases Java de manera controlada, para poder evaluar si el funcionamiento de cada uno de los métodos de la clase se comporta como se espera.
- En función de algún valor de entrada se evalúa el valor de retorno esperado
- Si la clase cumple con la especificación, entonces JUnit devolverá que el método de la clase pasó exitosamente la prueba
- En caso de que el valor esperado sea diferente al que regresó el método durante la ejecución, JUnit devolverá un fallo en el método correspondiente

Ejecución de JUnit

- Hasta Versión **JUnit3:**
  - Definir clases que hereden de la clase junit.framework.TestCase de Junit.
  - Implementar métodos testXXX() (sin parámetros), que serán los que hagan las pruebas necesarias.
  - Inicialización del test en el método setUp() para colocar dicho código que se ejecutará antes de realizar cada prueba.
  - Terminación del test en el método tearDown() en el caso de que queramos liberar algunos recursos después de cada prueba.
  - Clases AssertXXX para comprobación de los resultados: AssertTrue
  - Lanzamiento de conjunto de tests mediante método suite():

Ejecución de JUnit

- Versión **JUnit4:**
  - Incluye anotaciones (Java 5 annotations) en lugar de utilizar herencia:
  - @Test sustituye a la herencia de TestCase.
  - @Before y @After como sustitutos de setUp y tearDown.
  - @BeforeClass y @AfterClass, metodos estáticos para inicializar y liberar recursos en el Test

Ejecución de JUnit

- Versión **JUnit5:**
  - Permite trabajar con todas las novedades incluidas en Java 8:
    - Programación funcional
    - Lambdas
    - Streams, …etc.

Principales anotaciones en Junit 5

- **@Test**: Indica que un método es un caso de prueba.
- **@DisplayName**: Permite definir un nombre personalizado para una clase de prueba o un método de prueba, mejorando la legibilidad de los resultados.
- **@BeforeEach**: Indica que el método anotado se ejecutará antes de cada método de prueba (reemplaza a @Before de JUnit 4).
- **@AfterEach**: Indica que el método anotado se ejecutará después de cada método de prueba (reemplaza a @After de JUnit 4).
- **@BeforeAll**: Indica que el método anotado se ejecutará una vez antes de todos los métodos de prueba en la clase (reemplaza a @BeforeClass).
- **@AfterAll**: Indica que el método anotado se ejecutará una vez después de todos los métodos de prueba en la clase (reemplaza a @AfterClass).

Principales anotaciones en Junit 5

- **@Nested**: Indica que la clase anotada es una clase de prueba anidada y no estática.
- **@Tag**: Declara etiquetas para filtrar pruebas.
- **@Disabled**: Desactiva temporalmente una prueba (reemplaza a @Ignore de JUnit 4).
- **@TestFactory**: Denota un método que es una fábrica de pruebas para pruebas dinámicas.
- **@ParameterizedTest**: Indica que un método es una prueba parametrizada.
- **@RepeatedTest**: Permite crear pruebas que se ejecutan varias veces de forma automática.

Principales métodos de Aserciones en Junit 5

- En JUnit 5, los métodos de aserción son fundamentales para validar los resultados de las pruebas unitarias. Estos métodos están disponibles en la clase estática org.junit.jupiter.api.Assertions. A continuación, se describen los principales métodos de aserción:

1. **assertEquals(expected, actual):**
   - Verifica que el valor esperado sea igual al valor actual.
   - Soporta un tercer argumento opcional para un mensaje personalizado y un parámetro delta para comparaciones con valores decimales.

```
assertEquals(5, 2 + 3, "Los valores no son iguales");
```

2. **assertNotEquals(expected, actual):**
   - Verifica que dos valores sean diferentes.

Principales métodos de Aserciones en Junit 5

3. **assertTrue(condition) / assertFalse(condition):**
   - Verifica que una condición sea verdadera o falsa respectivamente.

```
assertTrue(5 > 3, "La condición no es verdadera");
```

4. **assertNull(object) / assertNotNull(object):**
   - Verifica que un objeto sea nulo o no nulo.

```
assertNotNull(myObject, "El objeto no debe ser nulo");
```

Principales métodos de Aserciones en Junit 5

5. **assertAll(heading, executable...):**
   - Agrupa múltiples aserciones y ejecuta todas, incluso si alguna falla.

```
assertAll("Validaciones",
    () -> assertEquals(10, 5 + 5),
    () -> assertNotNull("Texto")
);
```

6. **assertThrows(expectedType, executable):**
   - Verifica que se lance una excepción específica durante la ejecución de un bloque de código.

```
assertThrows(IllegalArgumentException.class, () -> {
    throw new IllegalArgumentException("Error");
});
```

Principales métodos de Aserciones en Junit 5

7. **assertIterableEquals(expected, actual):**
   - Compara dos objetos Iterable para verificar que sean iguales.

8. **assertArrayEquals(expectedArray, actualArray):**
   - Compara dos arreglos para verificar que sean iguales.

9. **assertLinesMatch(expectedLines, actualLines):**
   - Verifica que dos listas de cadenas coincidan línea por línea.

10. **fail(message):**
    - Marca explícitamente una prueba como fallida con un mensaje personalizado.

Mock Object

- Un Mock Object es un "objeto falso", un objeto que representa a otro y lo sustituye en funcionalidad. Este patrón es utilizado ampliamente en la Prueba Unitaria para asegurar un correcto aislamiento de la clase bajo test.

- Así, las dependencias que tenga nuestro objeto a testear pueden ser reemplazadas por mocks que funcionen como nosotros queremos. De lograr esto, podremos testear en forma aislada a nuestra clase, sin preocuparnos por sus dependencias (más aún, sin preocuparnos por si realmente funcionan estas dependencias).

- Usando Mock Objects podemos asegurar un "entorno perfecto y a medida", haciendo que este entorno responda como nosotros necesitamos. Luego, si el test de la clase falla, será por un problema en esta misma clase (y no en sus dependencias ya que, por hipótesis, el entorno era ideal).

Frameworks de Mocks

- Existen varios frameworks que ayudan a la creación de mocks:
  - EasyMock
  - Mockito
  - MockEjb

- Mockito es uno de los más conocidos.
- EasyMock es el usado por el equipo de Spring para testear su framework.

Mockito

- Mockito es una librería Java para la creación de Mock Object muy usados para el testeo unitario
- Mockito fue creado con el objetivo de simplificar y solucionar algunos de los temas antes mencionados.
- EasyMock y Mockito puede hacer exactamente las mismas cosas, pero Mockito tiene un API más natural y práctico de usar.

Principales anotaciones de Mockito

- Mockito es un framework de pruebas unitarias en Java que permite simular objetos y comportamientos. Sus anotaciones simplifican la creación, configuración e inyección de mocks en los tests. A continuación, se describen las principales anotaciones:
- **@Mock:**
  - Crea un objeto simulado (mock) de una clase o interfaz.
  - Se utiliza para reemplazar dependencias reales con simulaciones en las pruebas.

```
@Mock
private UsuarioRepositorio usuarioRepositorio;
```

Principales anotaciones de Mockito

- **@InjectMock:**
  - Crea una instancia del objeto que se está probando e inyecta automáticamente los mocks creados con @Mock o @Spy en sus dependencias.

```
@InjectMocks
private UsuarioServicio servicio;
```

- **@Spy:**
  - Envuelve un objeto real para espiar su comportamiento, permitiendo llamar a métodos reales y simular otros.

```
@Spy
private List<String> lista = new ArrayList<>();
```

Principales anotaciones de Mockito

- **@Captor:**
  - Crea un captor para capturar argumentos pasados a métodos simulados.

```
@Captor
private ArgumentCaptor<String> captor;
```

- **@MockBean:**
  - Reemplaza un bean real por un mock dentro del contexto de Spring.

```
@MockBean
private Repositorio repositorioMock;
```

Principales anotaciones de Mockito

- **@ExtendWith(MockitoExtension.class):**
  - Habilita el uso de anotaciones de Mockito en JUnit 5, inicializando automáticamente los mocks y spies.

```
@ExtendWith(MockitoExtension.class)
class MiTest {
    @Mock
    private MiDependencia dependencia;
}
```

Principales métodos de Mockito

- Mockito ofrece una variedad de métodos para crear y manejar objetos simulados (mocks) en pruebas unitarias. A continuación, se describen los métodos más importantes:
- **mock(Class<T> classToMock):**
  - Crea un mock de una clase o interfaz específica.

```
List<String> mockList = Mockito.mock(List.class);
```

- **when(T methodCall):**
  - Define el comportamiento esperado de un método simulado.

```
Mockito.when(mockList.size()).thenReturn(5);
```

Principales métodos de Mockito

- **thenReturn(value):**
  - Especifica el valor que debe devolverse cuando se llama al método simulado.

```
Mockito.when(mockList.get(0)).thenReturn("Hola");
```

- **verify(mock).method():**
  - Verifica si un método específico fue llamado en el mock, con los argumentos esperados.

```
Mockito.verify(mockList).add("Elemento");
```

- **doReturn(value).when(mock).method():**
  - Similar a when(), pero se utiliza cuando el método simulado no puede ser interceptado directamente (por ejemplo, métodos void).

```
Mockito.doReturn("Hola").when(mockList).get(0);
```

Principales métodos de Mockito

- **doNothing().when(mock).method():**
  - Define que no se haga nada cuando se llame a un método void.

```
Mockito.doNothing().when(mockList).clear();
```

- **doThrow(exception).when(mock).method():**
  - Configura un mock para lanzar una excepción específica cuando se llama a un método.

```
Mockito.doThrow(new RuntimeException()).when(mockList).clear();
```

- **assertThatThrownBy(() -> ...) (junto con assertThrows):**
  - Verifica que un método lance una excepción esperada.

Principales métodos de Mockito

- **reset(mock):**
  - Restablece un mock para eliminar configuraciones previas.

```
Mockito.reset(mockList);
```

- **spy(T object):**
  - Crea un objeto espía que permite llamar a métodos reales y simular otros.

```
List<String> spyList = Mockito.spy(new ArrayList<>());
```

- **times(n) / atLeast(n) / never() (usados con verify):**
  - Verifican la cantidad de veces que un método fue invocado.

```
Mockito.verify(mockList, Mockito.times(2)).add("Elemento");
```

ibertech

end2end dedication

Tema 4

Test avanzados

Test avanzados

- Los tests avanzados en software son aquellos que van más allá de las pruebas básicas (unitarias o funcionales) y buscan validar aspectos más complejos como rendimiento, seguridad, comportamiento bajo carga o resiliencia del sistema. Tipos de test avanzados:

    1. Pruebas de rendimiento
    2. Pruebas de seguridad
    3. Pruebas de automatización avanzada
    4. Pruebas de regresión avanzadas
    5. Pruebas de aceptación avanzadas (BDD)
    6. Pruebas de resilencia
    7. Pruebas de compatibilidad

1. Pruebas de rendimiento

- Evalúan cómo responde el sistema en términos de velocidad, estabilidad y escalabilidad.
    - Load testing: comportamiento con muchos usuarios
    - Stress testing: hasta que el sistema falla
    - Spike testing: picos repentinos de carga
- Herramientas:
    - Apache JMeter
    - Gatling

2. Pruebas de seguridad

- Detectan vulnerabilidades en el sistema:
    - Inyección SQL
    - Cross-Site Scripting (XSS)
    - Fallos de autenticación
- Herramientas:
    - OWASP ZAP
    - OWASP

3. Pruebas de automatización avanzada

- Van más allá de simples tests automatizados e incluyen:
    - Integración en CI/CD
    - Ejecución paralela
    - Tests en múltiples entornos
- Herramientas:
    - Selenium
    - Playwright

4. Pruebas de regresión avanzadas

- Verifican que los cambios no rompen funcionalidades existentes.
    - Se ejecutan automáticamente tras cada cambio
    - Suelen integrarse en pipelines de integración continua

5. Pruebas de aceptación avanzadas (BDD)

- Se basan en el comportamiento esperado del usuario.
- Ejemplo:
    - Especificaciones en lenguaje natural (Given-When-Then)
- Herramientas:
    - Cucumber

6. Pruebas de resilencia

- Evalúan cómo responde el sistema ante fallos inesperados:
    - Caída de servidores
    - Fallos de red
    - Latencia artificial
- Ejemplo:
    - Chaos Monkey

7. Pruebas de compatibilidad

- Verifican que el sistema funcione correctamente en:
  - Diferentes navegadores
  - Sistemas operativos
  - Dispositivos

ibertech

end2end dedication

Tema 5

TDD – Test Driven Development

Test Driven Development (TDD)

- Desarrollo guiado por pruebas, o Test Driven Development (TDD) es una práctica de programación que involucra otras dos prácticas:
  - Escribir las pruebas primero (Test First Development)
  - Refactorización (Refactoring).

- Para escribir las pruebas generalmente se utiliza la Prueba Unitaria

Test Driven Development (TDD)

- TDD (Test-Driven Development) es una metodología de desarrollo de software que consiste en escribir primero las pruebas unitarias antes de implementar el código de la funcionalidad. Este enfoque sigue un ciclo iterativo conocido como "rojo-verde-refactorizar":
  - **Fase roja**: Se escribe una prueba que inicialmente falla.
  - **Fase verde**: Se implementa el código necesario para que la prueba pase.
  - **Fase de refactorización**: Se optimiza el código sin cambiar su comportamiento.

- El objetivo principal de TDD es crear un código más robusto, modular y de mayor calidad. Al escribir las pruebas primero, los desarrolladores se centran en una única característica a la vez, lo que mejora la arquitectura de la solución y facilita la detección temprana de errores.

Ventajas TDD

- **Mejora la calidad del código**: Al escribir pruebas antes de implementar la funcionalidad, se asegura que cada línea de código esté justificada y cumpla con los requisitos establecidos.
- **Código más simple y menos redundante**: El ciclo constante de refactorización promueve la eliminación de código innecesario y la reutilización de funciones existentes.
- **Aumenta la productividad**: Se reduce el tiempo dedicado a la depuración, ya que los errores se detectan y corrigen tempranamente en el proceso de desarrollo.
- **Facilita la escalabilidad:** El enfoque modular del TDD permite crear código más fácil de mantener y expandir a medida que el proyecto crece.
- **Minimiza errores**: La detección temprana de problemas reduce significativamente el número de errores que llegan a producción.

Ventajas TDD

- **Mejora el diseño del software**: El proceso iterativo de TDD fomenta un diseño emergente y más efectivo de la arquitectura del software.
- **Proporciona documentación actualizada**: Los propios tests sirven como documentación ejecutable del comportamiento esperado del código.
- **Facilita la refactorización**: Permite mejorar el código existente sin temor a introducir nuevos errores, ya que las pruebas garantizan que la funcionalidad se mantiene.
- **Mejora la comunicación en el equipo**: Los problemas se abordan en períodos cortos de tiempo, fomentando la colaboración entre los miembros del equipo.
- **Aumenta la confianza en el código**: La alta cobertura de pruebas que se logra con TDD proporciona mayor seguridad sobre la funcionalidad del código en diversos escenarios.

El algoritmo TDD

- Escribir la prueba. Para escribir la prueba, el desarrollador debe entender claramente las especificaciones y los requisitos. El diseño del documento deberá cubrir todos los escenarios de prueba y condición de Excepciones.
- Escribir el código haciendo que pase la prueba. Este paso fuerza al programador a tomar la perspectiva de un cliente considerando el código a través de sus interfaces. Ésta es la parte conducida por el diseño, del TDD. Como parte de la calibración de la prueba, el código debe fallar la prueba significativamente las primeras veces.
- Ejecutar las pruebas automatizadas. Si pasan, el programador puede garantizar que el código resuelve los casos de prueba escritos. Si hay fallos, el código no resolvió los casos de prueba.
- Refactorización y limpieza en el código. Después se vuelven a efectuar los casos de prueba y se observan los resultados.
- Repetición. Después se repetirá el ciclo y se comenzará a agregar las funcionalidades adicionales o a arreglar cualquier error.

ibertech

end2end dedication

Tema 6

BDD – Behavior Driven Development

Behavior Driven Development (BDD)

- BDD en testing significa "Behavior Driven Development" o "Desarrollo guiado por comportamiento" y es una metodología ágil que busca mejorar la colaboración entre desarrolladores, testers y el área de negocio usando pruebas escritas en lenguaje natural.

- BDD es un proceso de desarrollo de software que surge como evolución del TDD (Test Driven Development). Mientras que el TDD se enfoca en pruebas unitarias para las funcionalidades técnicas, el BDD redefine las pruebas poniéndolas en términos de comportamientos esperados desde la perspectiva del usuario o cliente final. Esto significa que las pruebas se describen usando ejemplos reales y lenguaje comprensible para todos los involucrados en el proyecto

BDD vs TDD

- BDD se orienta al comportamiento del sistema y a la experiencia del usuario, mientras que TDD se enfoca exclusivamente en pruebas técnicas de código. Ambas metodologías pueden coexistir, pero BDD fomenta mayor integración y participación de todos los equipos, no sólo los desarrolladores.

- Un escenario en BDD podría ser:
  - Dado que el usuario no ha introducido ningún dato en un formulario
  - Cuando hace clic en "Enviar"
  - Entonces debe mostrar un mensaje de validación.

- Esta estructura facilita que todos los miembros del equipo comprendan qué se espera del sistema ante determinadas acciones.

ibertech

end2end dedication

Muchas gracias por vuestra asistencia

www.ibertech.es